hashlock.

Security Audit

CeReBree (HRTech)

September 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

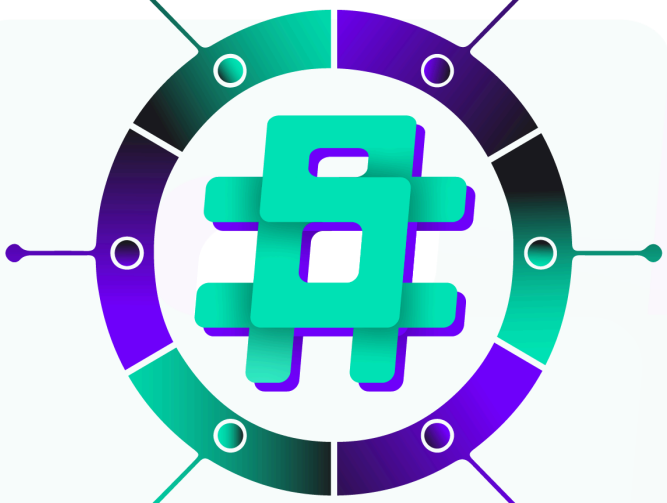
The CeReBree team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

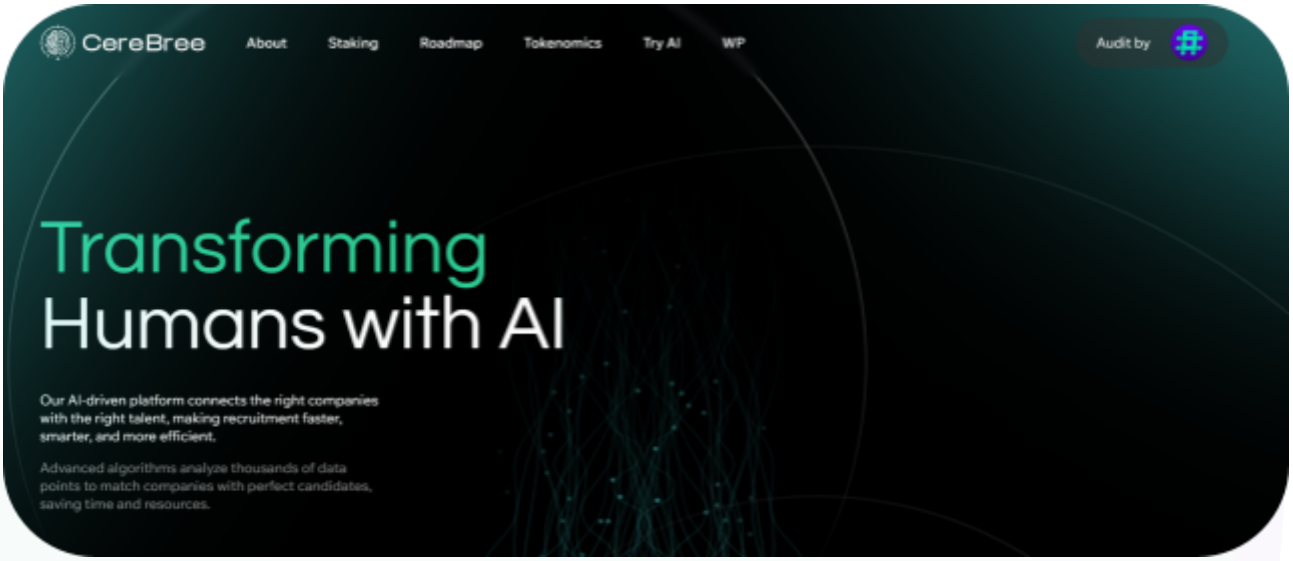
Cerebree is a blockchain-enabled AI recruitment platform aimed at transforming how companies and candidates match. It leverages advanced algorithms to align not just skills but also cultural fit and personality. Token holders gain benefits like early access, revenue sharing, and governance rights, indicating a decentralized model. The site presents tokenomics, staking, and a roadmap showing stages from early launch to global scaling. Its target audience spans recruiters, job seekers, and investors in AI + blockchain ecosystems.

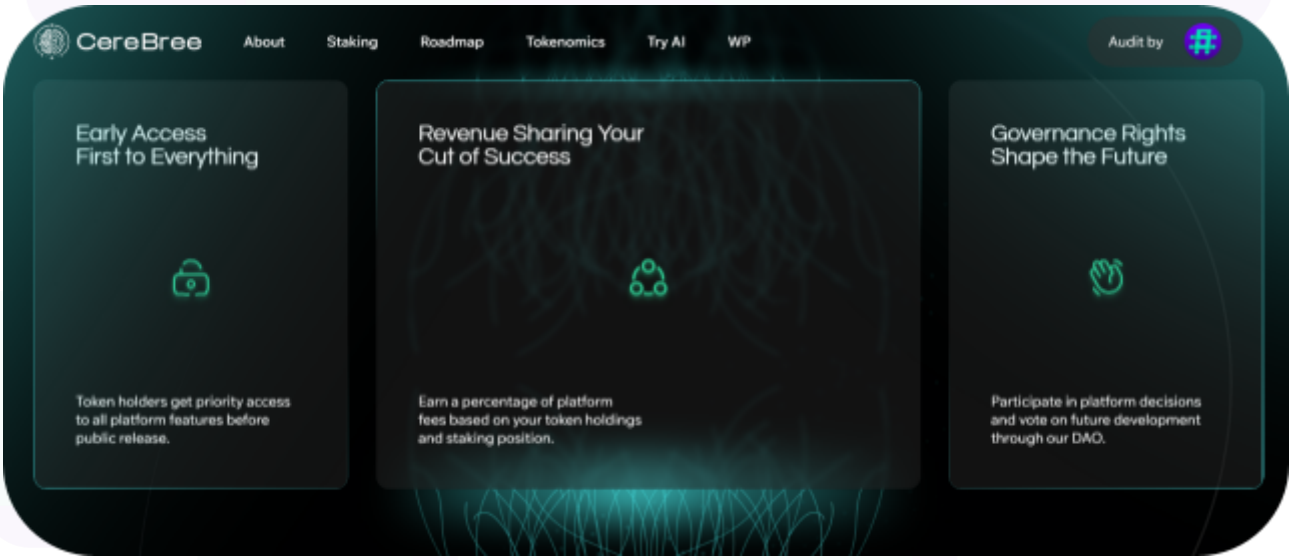
Project Name: CeReBree
Project Type: HRTech
Compiler Version: ^0.8.30
Website: https://cerebree.icoda.io/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the CeReBree project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**CeReBree Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**September, 2025**
Contract 1	Presale.sol
Contract 2	Staking.sol
Contract 3	Token.sol
Contract 4	IPresale.sol
Contract 5	IStaking.sol
Audited GitHub Commit Hash	b6d6c5452f701a9f91adee79bcf2ca22defc5566
Fix Review GitHub Commit Hash	f85b5c97824eab430d93d50261e74fe4a1033476

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section, and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

5 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Presale.sol - Interface for CeReBree token presale contract - Supports ETH and USDT payments with Chainlink price feeds for USD conversion - Admin functions to set the token price in USD (18 decimals), the minimum purchase amount, and the presale start time - Public functions allow users to purchase tokens with ETH or USDT - Enforces minimum purchase requirements in USD - Withdraws collected ETH and tokens to the owner	**Contract achieves this functionality.**
Staking.sol - Multi-tier staking system with time-locked positions and reward calculation - Three duration options: 1 month (8% APR), 3 months (12% APR), 6 months (18% APR) - Allows batch unstaking of multiple positions for gas efficiency - Tracks total allocated funds to prevent over-commitment	**Contract achieves this functionality.**

Token.sol - ERC20 token implementation named "CeReBree" with symbol "CRB" - Fixed maximum supply of 1,000,000,000 tokens (1 billion) - Includes burnable functionality through ERC20Burnable extension - All tokens are minted to the deployer address at construction - No additional minting capability after deployment - Uses 18 decimals (standard ERC20)	**Contract achieves this functionality.**
IPresale.sol - Defines an interface for the Presale contract - Declares functions, events, and errors	**Contract achieves this functionality.**
IStaking.sol - Defines an interface for the Staking contract - Declares functions, events, errors, and data structures	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the CeReBree project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices, and to help avoid unnecessary complexity that increases the likelihood of exploitation, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the CeReBree project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] Presale#constructor - Missing Zero Address Validation

Description

The Presale contract constructor does not validate that the input address parameters are not zero addresses before assigning them to state variables.

Vulnerability Details

The constructor accepts three address parameters (`_token`, `_usdt`, `_priceFeed`) and directly assigns them to immutable state variables without validating they are not `address(0)`. Since there are no setter functions to update these variables after deployment, passing a zero address would result in a permanently broken contract that cannot be fixed without redeployment.

```
constructor(
   Token _token,
   IERC20 _usdt,
   AggregatorV3Interface _priceFeed
) Ownable(msg.sender) {
   token = _token;
   usdt = _usdt;
   priceFeed = _priceFeed;
}
```

If any of these addresses are set to `address(0)`, critical functions will fail:

1- `buyTokensWithETH()` will revert when calling `token.balanceOf()` or `token.transfer()`

2- `buyTokensWithUSDT()` will revert when `calling usdt.transferFrom()` or `token.transfer()`

3- `_getETHToUSDPrice()` will revert when calling `priceFeed.latestRoundData()` or `priceFeed.decimals()`

Impact

While this is primarily a deployment-time issue rather than a runtime exploit, it represents a defensive programming gap. An accidental deployment with incorrect parameters would result in a permanently unusable contract, wasted deployment gas, and potential confusion or delays in the presale launch. This could be particularly problematic if the contract address has already been announced to users.

Recommendation

We recommend adding zero address validation checks in the constructor function to prevent accidental deployment with invalid addresses.

Status

Resolved

[L-02] Presale#withdrawToken - Unrestricted Token Withdrawal During Active Presale

Description

The `withdrawToken` function allows the owner to withdraw any `ERC20` tokens from the contract at any time, including the presale tokens and collected `USDT`/payment tokens, without validating whether the presale is still active or has concluded.

Vulnerability Details

The function lacks presale status validation, enabling the owner to withdraw critical tokens during an active presale. This creates a centralization risk where the owner could withdraw all presale tokens while sales are ongoing, preventing future buyers from completing purchases. While this is an owner-privileged function, the absence of presale state checks means there are no technical guardrails preventing withdrawal of tokens that should remain locked until presale completion.

```
function withdrawToken(IERC20 _token) external override onlyOwner {
    uint256 balance = _token.balanceOf(address(this));
    if (balance == 0) {
        revert NoTokensToWithdraw();
    }
    _token.transfer(msg.sender, balance);
    emit TokensWithdrawn(msg.sender, balance, address(_token));}
```

Impact

This design introduces unnecessary centralization risk and could damage user confidence. If the owner withdraws presale tokens during an active sale, subsequent purchase transactions will fail, creating a poor user experience. Users have no technical guarantee that tokens will remain available until the presale ends, requiring complete trust in the owner's intentions.

Recommendation

We recommend implementing presale status validation to prevent withdrawal of critical tokens during active presale periods. Add checks to ensure presale tokens and collected `USDT` cannot be withdrawn while the presale is ongoing, providing users with stronger guarantees and reducing centralization concerns.

Status

Resolved

[L-03] Presale#buyTokensWithETH - Missing Zero Address Validation for Receiver

Description

The `buyTokensWithETH` function lacks zero address validation for the `_receiver` parameter, allowing tokens to be permanently lost if sent to `address(0)`.

Vulnerability Details

The function accepts a `_receiver` address without validation. Since `ERC20's transfer()` does not revert when sending to `address(0)`, tokens are effectively burned while the user's `ETH` payment remains captured by the contract, resulting in permanent loss.

```
function buyTokensWithETH(address _receiver) public payable override {
    if (!_hasPresaleStarted()) {
        revert PresaleNotStarted(); }
    uint256 ethToUSDPrice = _getETHToUSDPrice();
    uint256 purchaseValueInUSD = (msg.value * ethToUSDPrice) / 1 ether;
    uint256 tokensToBuy = (purchaseValueInUSD * 1 ether) / tokenPriceUSD;
    if (purchaseValueInUSD < minPurchaseUSD) {
        revert MinimumPurchaseNotMet(); }
    uint256 contractTokenBalance = token.balanceOf(address(this));
    if (contractTokenBalance < tokensToBuy) {
        revert NotEnoughTokensForSale(); }
    token.transfer(_receiver, tokensToBuy);
}
```

Impact

Users can permanently lose purchased tokens if `address(0)` is specified as the receiver, while their `ETH` payment remains captured by the contract. This creates an asymmetric loss where users lose both tokens and payment with no recovery mechanism available.

Recommendation

We recommend adding zero address validation at the function's beginning to prevent accidental token burns. Include `if (_receiver == address(0)) revert`

`InvalidReceiverAddress()`; before processing the purchase to ensure tokens are only sent to valid addresses.

Status

Resolved

[L-04] Presale#buyTokensWithUSDT - Missing Zero Address Validation for Receiver

Description

The `buyTokensWithUSDT` function does not validate that the `_receiver` parameter is not the zero address before transferring tokens.

Vulnerability Details

The function accepts a receiver address parameter without checking if it equals `address(0)`. If a zero address is provided, tokens will be transferred to the burn address, resulting in permanent token loss while the user's `USDT` payment is collected by the contract.

```
function buyTokensWithUSDT(
    address _receiver,
    uint256 _usdtAmount
) public override {
    if (!_hasPresaleStarted()) {
        revert PresaleNotStarted(); }
    uint256 convertedUSDAmount = _convertUSDTToUSD(_usdtAmount);
    if (convertedUSDAmount < minPurchaseUSD) {
        revert MinimumPurchaseNotMet(); }
    uint256 tokensToBuy = (convertedUSDAmount * 1 ether) / tokenPriceUSD;
    if (token.balanceOf(address(this)) < tokensToBuy) {
        revert NotEnoughTokensForSale(); }
    usdt.transferFrom(msg.sender, address(this), _usdtAmount);
    token.transfer(_receiver, tokensToBuy); }
```

Impact

Users can permanently lose their purchased tokens if `address(0)` is accidentally passed as the receiver parameter. The `USDT` payment is collected successfully, while tokens are sent to an unrecoverable address, resulting in complete fund loss for the buyer.

Recommendation

We recommend adding a zero address validation check at the beginning of the function to revert transactions when `_receiver == address(0)`, preventing accidental token burns and protecting users from permanent fund loss.

Status

Resolved

[L-05] Staking#constructor - Missing Zero Address Validation

Description

The Staking contract constructor does not validate that the token address parameter is not the zero address before assigning it to the immutable state variable.

Vulnerability Details

The constructor accepts the token address and directly assigns it without validation. Since no setter function exists to update the token address after deployment, passing a zero address would result in a permanently broken contract, requiring redeployment and wasting gas.

```
constructor(IERC20 _token) Ownable(msg.sender) {
    token = _token;
}
```

If deployed with `address(0)`, all staking operations will fail:

Impact

An accidental deployment with zero address renders the contract permanently unusable. All staking functions would revert, requiring contract redeployment, resulting in wasted deployment gas and potential delays in the launch timeline.

Recommendation

Add zero address validation in the constructor to prevent deployment with an invalid token address. Implement a check that reverts if the token parameter equals address(0) before assignment.

Status

Resolved

QA

[Q-01] Presale#buyTokensWithETH - Missing Event Emission for Token Purchases

Description

The `buyTokensWithETH` function does not emit an event to log token purchase transactions.

Vulnerability Details

The function completes the purchase flow without emitting any event. This prevents off-chain systems from monitoring purchases, makes auditing total sales impossible, and removes transparency for users verifying their transactions.

```
function buyTokensWithETH(address _receiver) public payable override {
    token.transfer(_receiver, tokensToBuy);
}
```

Impact

The presale lacks transparency and monitoring capabilities. Users cannot verify purchases on blockchain explorers, backend systems cannot track sales metrics, and there's no on-chain audit trail for allocations.

Recommendation

Add a `TokensPurchased` event that logs buyer, receiver, `ETH` amount, and token amount. Emit after successful token transfer.

Status

Resolved

[Q-02] Presale#buyTokensWithUSDT - Missing Event Emission for Token Purchases

Description

The `buyTokensWithUSDT` function does not emit an event to log token purchase transactions.

Vulnerability Details

The function completes the purchase flow without emitting any event. This prevents off-chain systems from monitoring purchases, makes auditing total sales impossible, and removes transparency for users verifying their transactions.

```
function buyTokensWithUSDT(
    address _receiver,
    uint256 _usdtAmount
) public override {
    usdt.transferFrom(msg.sender, address(this), _usdtAmount);
    token.transfer(_receiver, tokensToBuy);
}
```

Impact

The presale lacks transparency and monitoring capabilities. Users cannot verify purchases on blockchain explorers, backend systems cannot track sales metrics, and there's no on-chain audit trail for allocations.

Recommendation

Add a `TokensPurchased` event that logs buyer, receiver, `USDT` amount, and token amount. Emit after successful token transfer.

Status

Resolved

[Q-03] All Contracts - Floating Pragma Version

Description

All contracts use a floating pragma `^0.8.30` instead of locking to a specific compiler version.

Vulnerability Details

The contracts use pragma solidity `^0.8.30`, which allows compilation with any version from `0.8.30` up to (but not including) `0.9.0`. This creates deployment inconsistencies where different compiler versions may produce different bytecode, introducing subtle bugs or behavioral changes across deployments.

```
pragma solidity ^0.8.30;
```

Impact

Different compiler versions may introduce bugs or behavioral differences. Testing on one version doesn't guarantee the same behavior when deployed with another version, potentially causing unexpected issues in production deployments.

Recommendation

Lock the pragma to a specific compiler version using pragma solidity `0.8.30`; to ensure consistent compilation and deployment behavior across all environments.

Status

Resolved

[Q-04] Staking#_hasStakingStarted - Debug Code Left in Production

Description

The contract contains debug logging code from `Foundry's` testing framework that should be removed before deployment.

Vulnerability Details

The `_hasStakingStarted` function includes a console.log statement that was left from development. This executes on every stake call, wasting gas unnecessarily. The import statement for the testing library also remains in the code.

```
function _hasStakingStarted() internal view returns (bool) {
    console.log(block.timestamp, stakingStartTime);

    return block.timestamp >= stakingStartTime && stakingStartTime != 0;
}
```

Impact

Users pay extra gas for debug code that serves no purpose on the mainnet. This indicates the code needs final cleanup before production deployment.

Recommendation

Remove the import statement on line `9` and delete the console.log call on line `219` before deploying to production.

Status

Resolved

Centralisation

The CeReBree project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the CeReBree project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au